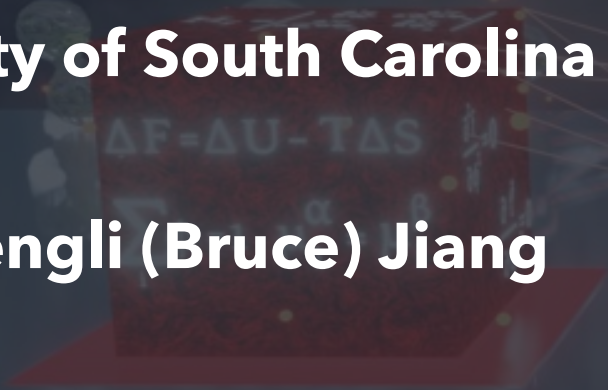


SciPy Curve Fitting and Uncertainty

Fall 2026

University of South Carolina

Prof. Shengli (Bruce) Jiang



The Maxwell Model for Stress Relaxation

How quickly does a polymer “forget” a deformation?

When a polymer is suddenly stretched and held at constant strain, its stress gradually decreases as the molecular chains rearrange.

$$\hat{G}(t; \boldsymbol{\theta}) = G_0 \exp\left(-\frac{t}{\tau}\right) \quad \boldsymbol{\theta} = \begin{bmatrix} G_0 \\ \tau \end{bmatrix}$$

- G_0 is the initial relaxation modulus (stiffness immediately after deformation)
- τ is the characteristic relaxation time
- At $t = \tau$, the modulus has fallen to $(\frac{G_0}{e} \approx 0.37G_0)$

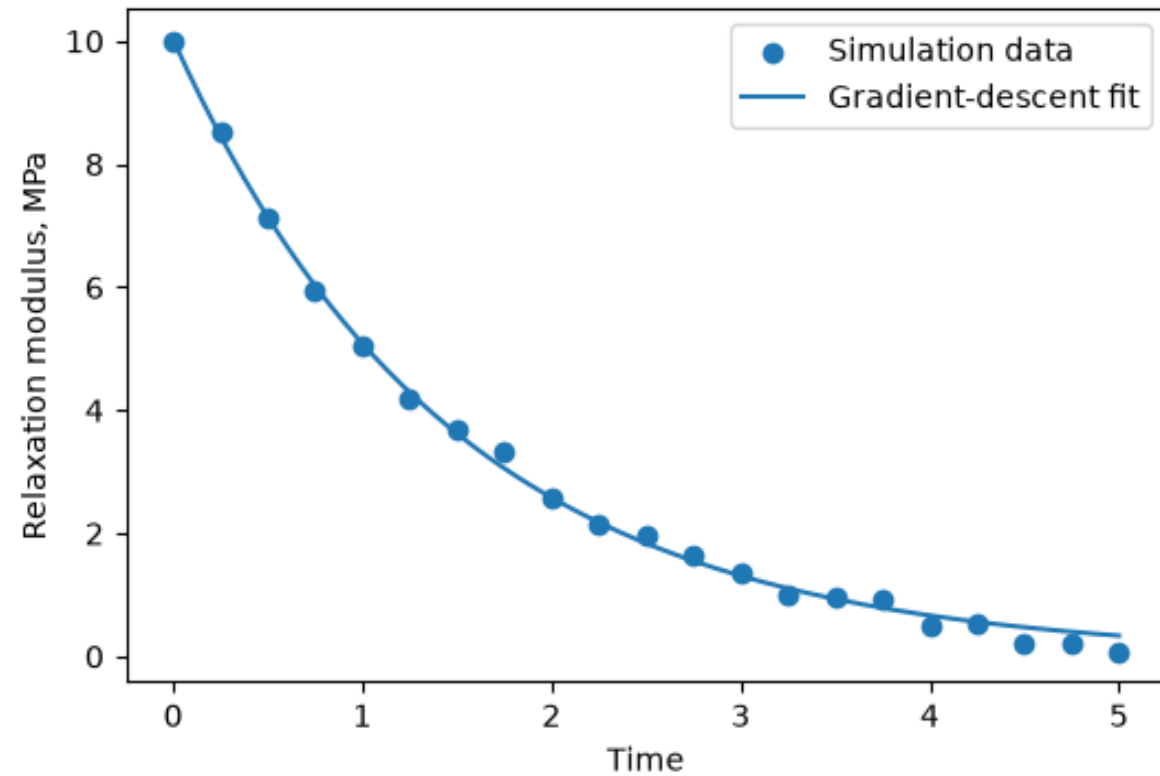


$$J(\boldsymbol{\theta}) = \frac{1}{2n} \sum_{i=1}^n \left[\hat{G}(t_i; \boldsymbol{\theta}) - G_i \right]^2$$

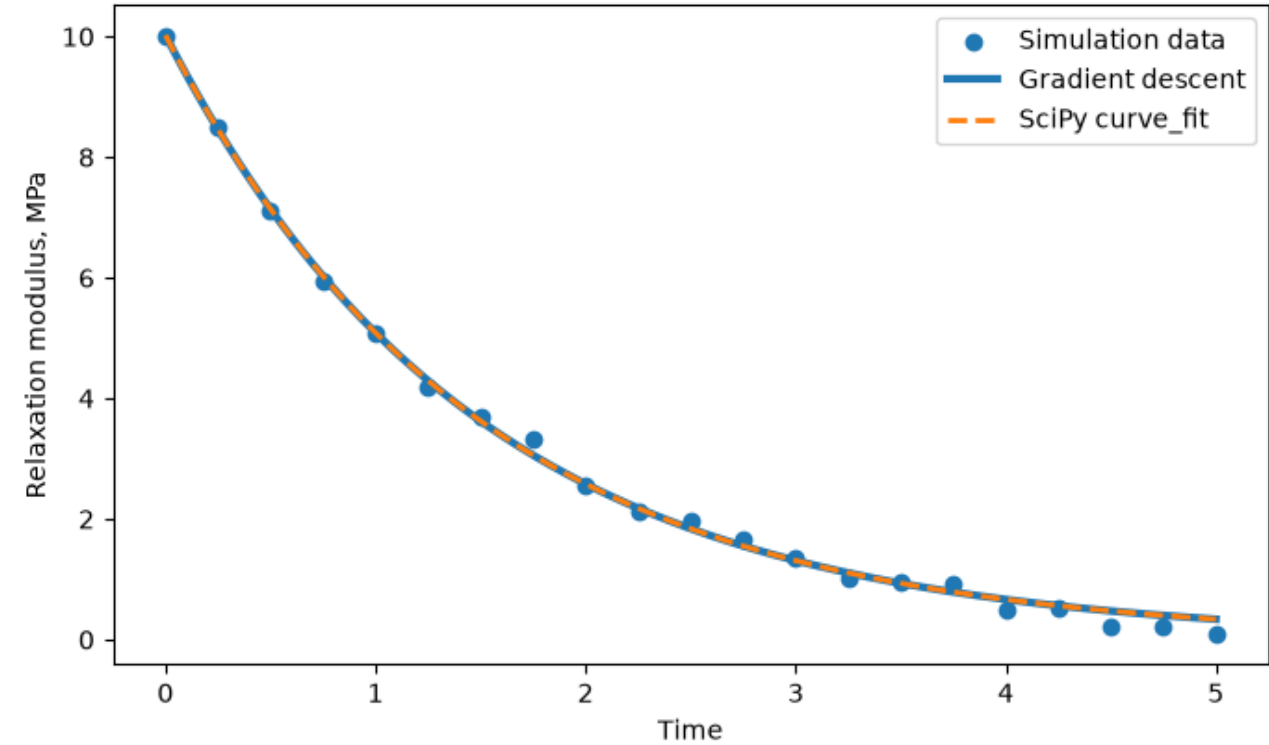
$$\frac{\partial \hat{G}}{\partial G_0} = \exp\left(-\frac{t}{\tau}\right) \quad \frac{\partial \hat{G}}{\partial \tau} = G_0 \exp\left(-\frac{t}{\tau}\right) \frac{t}{\tau^2}$$

Two Methods Can Produce the Same Fit

Maxwell Stress Relaxation



Nonlinear Fit of Polymer Stress Relaxation



Fitting the Maxwell Model with SciPy

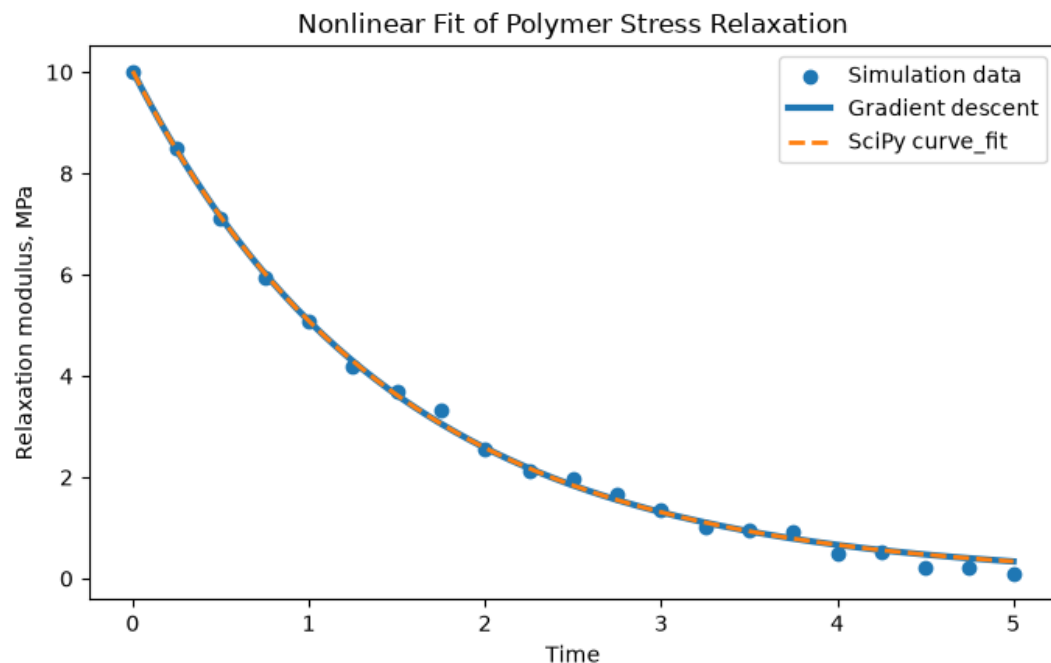
scipy.optimize.

curve_fit

`curve_fit(f, xdata, ydata, p0=None, sigma=None, absolute_sigma=False, check_finite=None, bounds=(-inf, inf), method=None, jac=None, *, full_output=False, nan_policy=None, **kwargs)` [\[source\]](#)

Use non-linear least squares to fit a function, f , to data.

Assumes $ydata = f(xdata, *params) + \epsilon$.



```
def maxwell_curve(t, G0, tau):  
    return G0 * np.exp(-t / tau)
```

```
theta_curve_fit, covariance = curve_fit(  
    maxwell_curve,  
    t,  
    G_simulation,  
    p0=theta_initial,  
    bounds=(0, np.inf),  
)
```

```
parameter_error = np.sqrt(np.diag(covariance))
```

```
print("G0", theta_curve_fit[0])  
print("tau", theta_curve_fit[1])  
print("G0 standard error", parameter_error[0])  
print("tau standard error", parameter_error[1])
```

```
G0 10.022390885765963  
tau 1.4739500453574912  
G0 standard error 0.0954923457837141  
tau standard error 0.021896329137659626
```

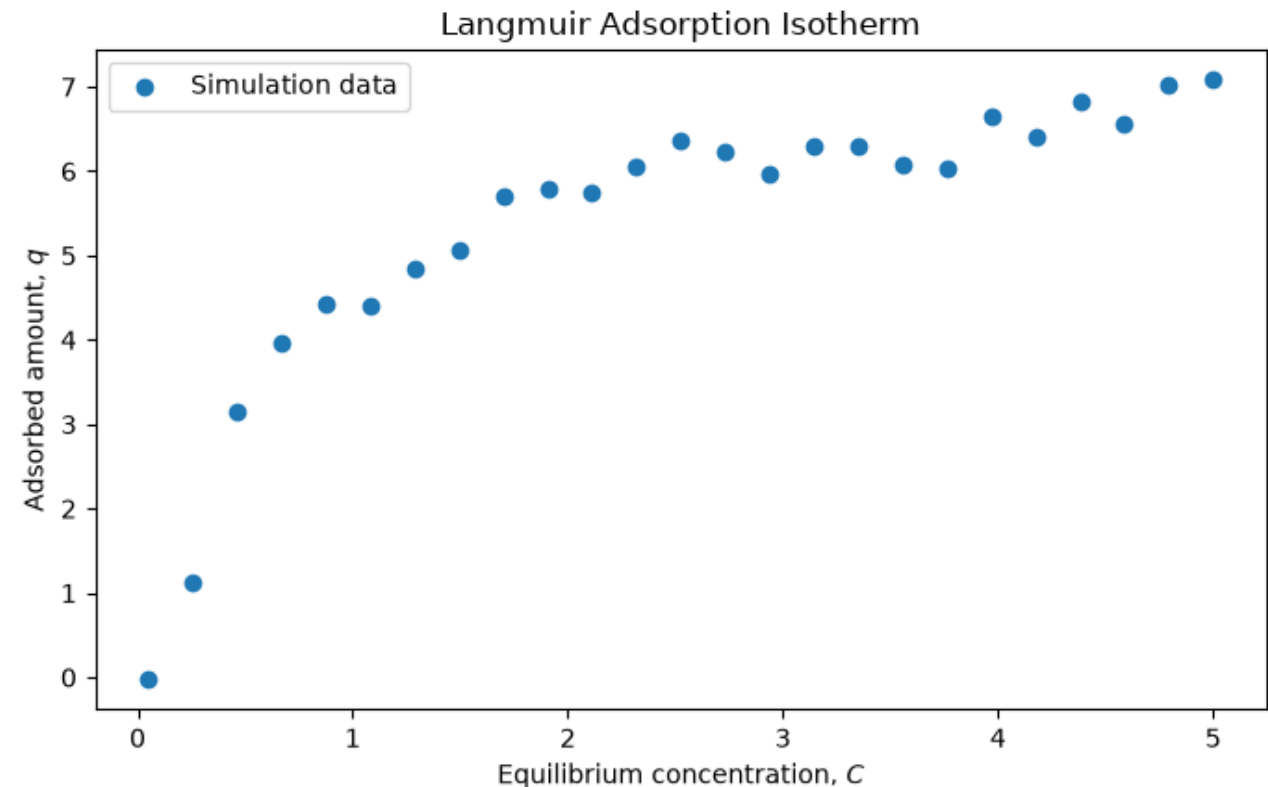
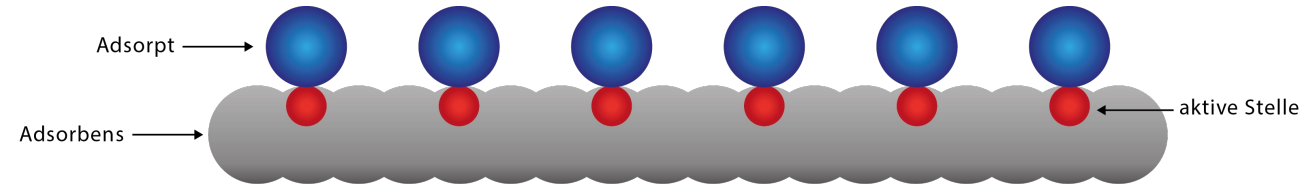
Example: The Langmuir Model for Saturating Adsorption

How does a surface fill up with molecules?

When molecules adsorb from a solution, they occupy a finite number of surface sites. Adsorption rises quickly at low concentration and approaches a plateau as the surface becomes saturated.

$$\hat{q}(C; \boldsymbol{\theta}) = \frac{q_{\max} K C}{1 + K C}, \quad \boldsymbol{\theta} = \begin{bmatrix} q_{\max} \\ K \end{bmatrix}$$

- $q_{\max}$: the maximum adsorption capacity.
- K : adsorption affinity.
- At $C = 1/K$, adsorption reaches $q_{\max}/2$.
- As $C \rightarrow \infty$, $q \rightarrow q_{\max}$.

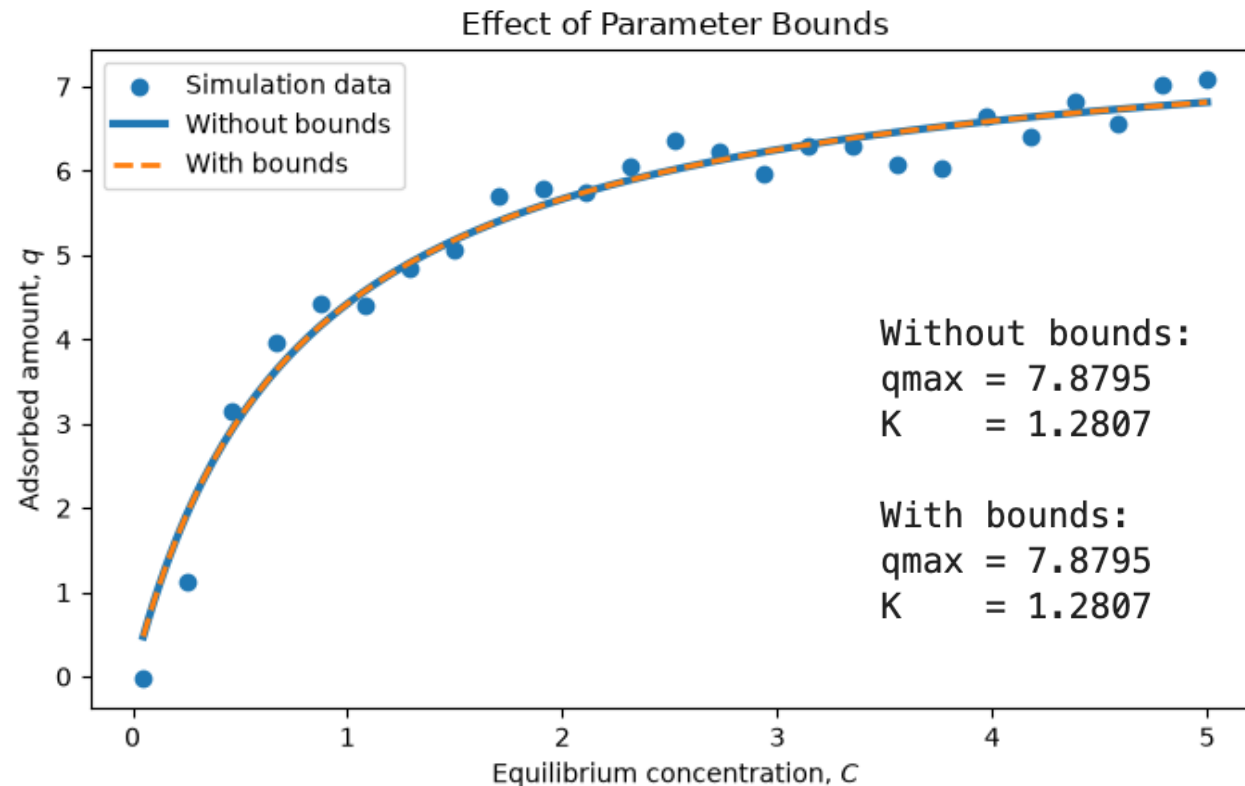


Physical Bounds Keep Parameters Meaningful

```
def langmuir(C, qmax, K):  
    return qmax * K * C / (1 + K * C)  
  
popt, pcov = curve_fit(  
    langmuir, C, q,  
    p0=[5, 0.5],  
    bounds=([0, 0], [np.inf, np.inf])  
)
```

- We already know the negative adsorption capacity makes no physical sense.
- $q_{\max} \geq 0, K \geq 0$.

```
print(popt)  
[7.87947663 1.28070895]
```



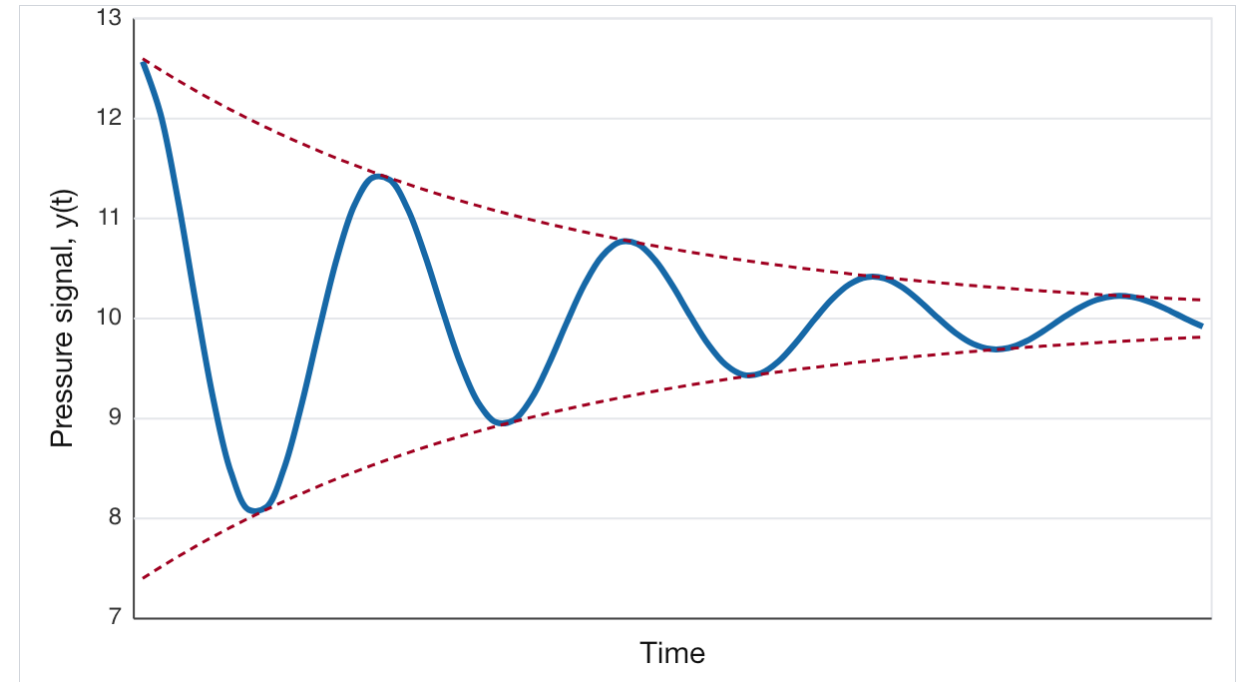
Example: Modeling a Damped Process Signal

Why does a disturbed process overshoot and oscillate before returning to steady state?

Energy storage and feedback create the oscillation. Dissipation removes energy, so each cycle becomes smaller.

$$y(t) = Ae^{-bt} \cos(\omega t + \phi) + c$$

- A : initial oscillation amplitude.
- b : damping rate; larger b means faster settling.
- ω : oscillation frequency
- ϕ : phase shift
- c : final steady-state value



Where we see it?

- Temperature, pressure, flow or level after a setpoint or load change
- Pressure fluctuations and bed motion in gas-solid fluidized beds
- Ring-down of pumps, compressors, piping, valves, and pressure sensors

Impact of Initial Guesses

```
def damped(t, A, b, omega, phi, c):  
    return A * np.exp(-b * t) * np.cos(omega * t + phi) + c
```

```
# Generate reproducible simulation data
```

```
t = np.linspace(0, 12, 100)  
rng = np.random.default_rng(26)
```

```
theta_true = [3.0, 0.10, 3.8, 0.45, 10.0]  
y_clean = damped(t, *theta_true)  
y = y_clean + rng.normal(0, 0.25, t.size)
```

```
# Parameter order: [A, b, omega, phi, c]
```

```
initial_guesses = {  
    "Good guess": [2.8, 0.12, 3.6, 0.30, 10.0],  
    "Low-frequency guess": [1.5, 0.30, 1.4, -0.50, 9.8],  
    "High-frequency guess": [1.5, 0.30, 6.2, 0.80, 10.2],  
}
```

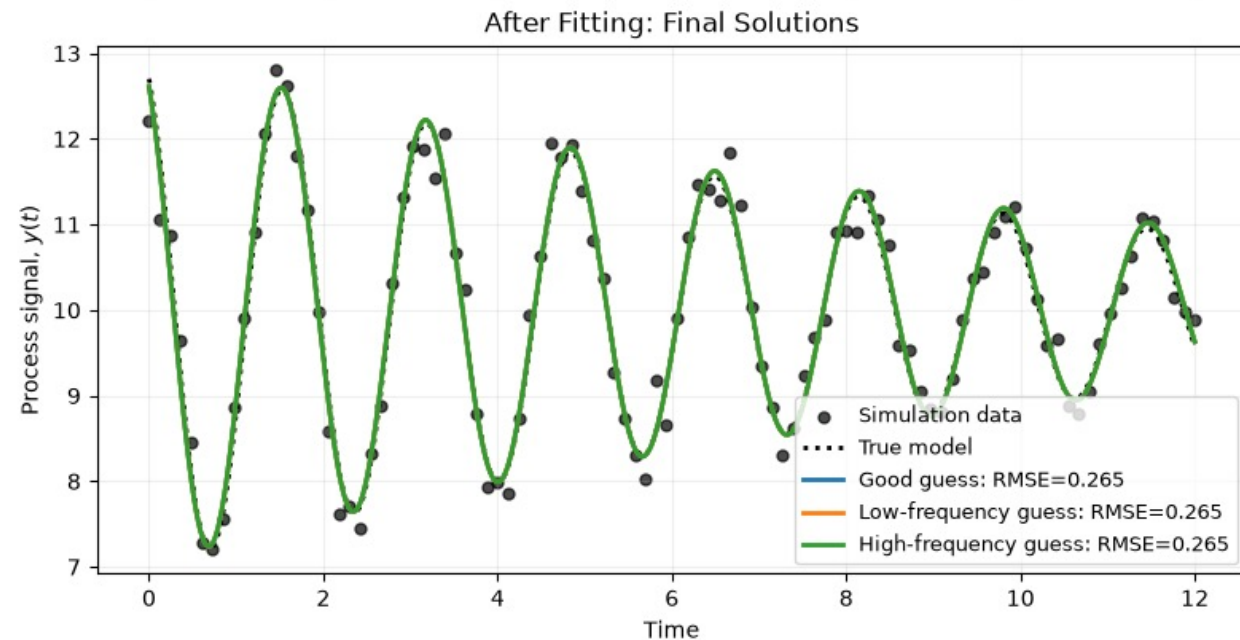
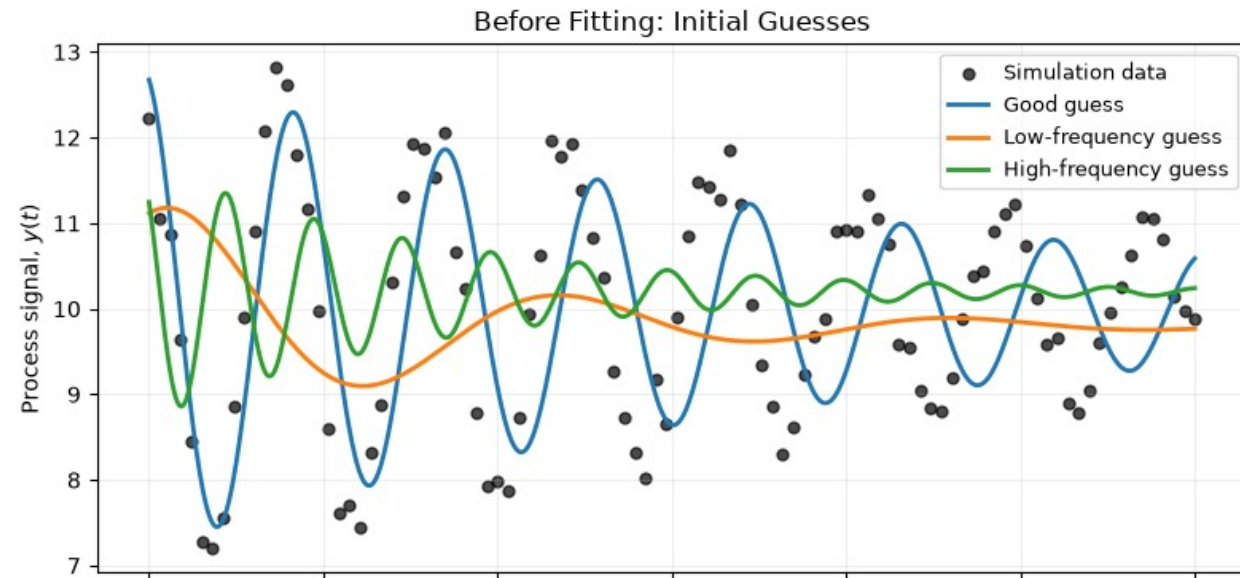
```
# Keep parameters within physically reasonable ranges
```

```
lower_bounds = [0, 0, 0, -2 * np.pi, -np.inf]  
upper_bounds = [np.inf, 2, 8, 2 * np.pi, np.inf]  
bounds = (lower_bounds, upper_bounds)
```

```
# Fit once from each initial guess
```

```
results = {}
```

```
for name, p0 in initial_guesses.items():  
    popt, pcov = curve_fit(damped, t, y, p0=p0, bounds=bounds, maxfev=30000)  
    residuals = y - damped(t, *popt)  
    rmse = np.sqrt(np.mean(residuals**2))  
    results[name] = {"p0": p0, "popt": popt, "pcov": pcov, "rmse": rmse}
```



Choosing an Optimization Method in SciPy

Method	Bounds?	When to use it?
lm	No	Small unconstrained problems
trf	Yes	Good default when bounds are needed
dogbox	Yes	Another option for small bounded problems

SciPy normally uses `lm` without bounds and `trf` when bounds are supplied.
`lm` cannot handle bounds.

```
curve_fit(model, x, y, p0=p0, method="lm")  
curve_fit(model, x, y, p0=p0, bounds=bounds, method="trf")  
curve_fit(model, x, y, p0=p0, bounds=bounds, method="dogbox")
```

Do not turn this into a horse race.

For an easy problem, all three may give nearly the same curve.

A Best Fit Still Has Uncertainty

Labeled Data

$$\{(x, y)_i\}$$

Model

$$f(x; \theta)$$

Predictions

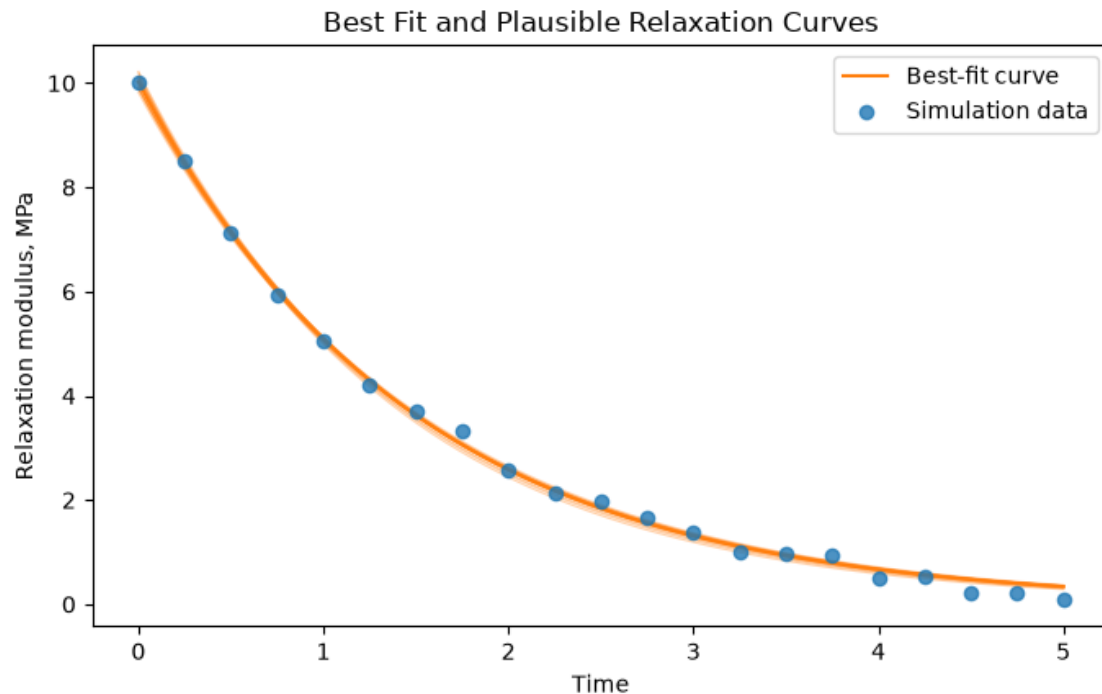
$$\hat{y}_i = f(x_i; \theta)$$

Loss

$$\mathcal{E}(y, \hat{y}, \theta, \dots)$$

optimization allows for selection of model parameters according to the loss

Accuracy and certainty are different. A best-fit parameter is not guaranteed to equal the true parameter.



$$\hat{\theta} = \begin{bmatrix} \hat{G}_0 \\ \hat{\tau} \end{bmatrix} = \begin{bmatrix} 10.022 \text{ MPa} \\ 1.474 \text{ s} \end{bmatrix}$$

- These values minimize the squared residuals.
- They are estimates obtained from one noisy experiment.
- Repeating the experiment would give slightly different values.

How much could $\hat{G}_0$ and $\hat{\tau}$ change if we repeated the experiment?

Measurement Noise Changes Parameter Estimates

We model each measurement as

$$G_i = G_0 \exp\left(-\frac{t_i}{\tau}\right) \boxed{+ \varepsilon_i}$$

**Inherent noise in our
observation or measurement**

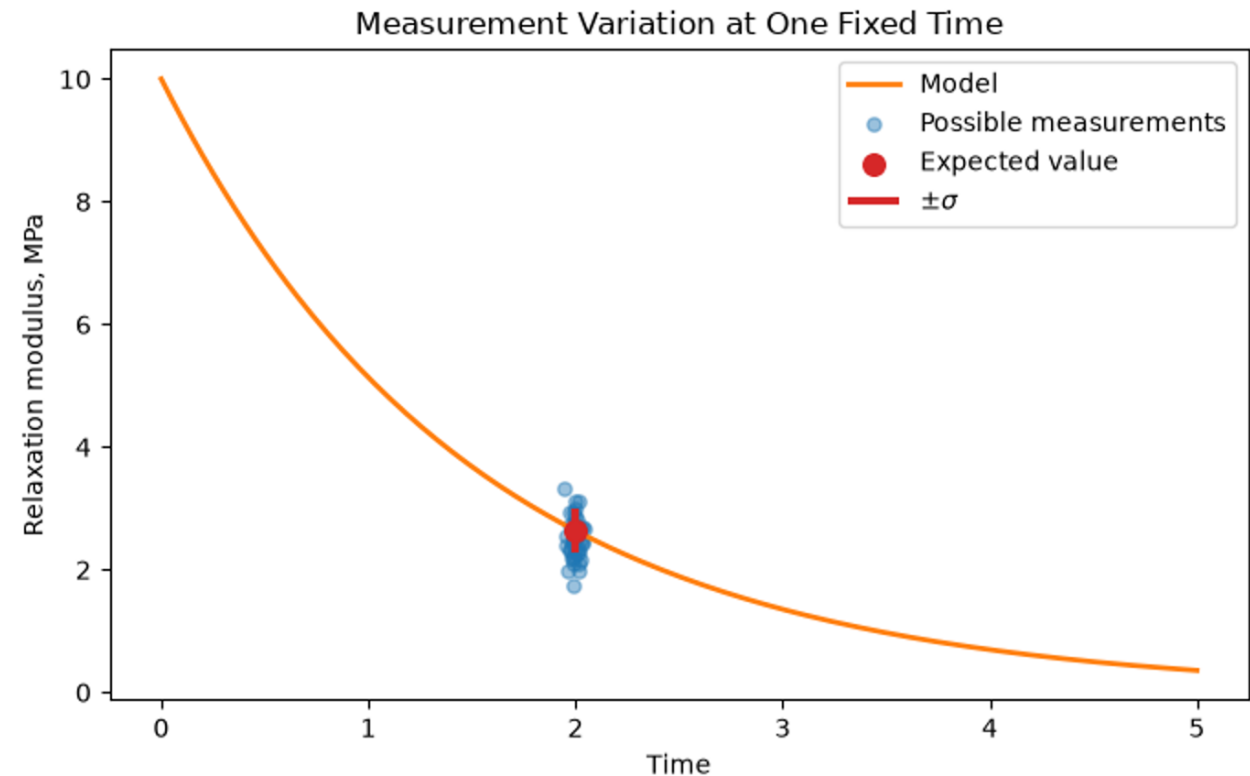
where

$$\mathbb{E}[\varepsilon_i] = 0, \quad \text{Var}(\varepsilon_i) = \sigma^2$$

- t_i : measurement time selected by the experimenter
- G_i : measured relaxation modulus
- ε_i : measurement and specimen variation
- σ : typical size of that variation

For now, we assume

$$\varepsilon_i \stackrel{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2)$$



Using Residuals to Estimate Measurement Noise

The residual for measurement i is

$$e_i = G_i - \hat{G}_0 \exp\left(-\frac{t_i}{\hat{\tau}}\right)$$

The residual sum of squares is

$$\text{SSE} = \sum_{i=1}^n e_i^2$$

Estimate the **noise variance** with

$$s^2 = \frac{\text{SSE}}{n - p}$$

$n - p$ is the amount of information left for estimating noise after fitting the model.

For this experiment:

$$n = 21, \quad p = 2, \quad n - p = 19$$

$$s^2 = \frac{\text{SSE}}{n - p} = \frac{0.347}{19} = 0.0183 \text{ MPa}^2$$

$$s = 0.135 \text{ MPa}$$

Measurements typically scatter about 0.135 Mpa above or below the fitted curve.

Why $n-p$?

- Each fitted parameter uses one degree of freedom.

Example

A line has two parameters: slope and intercept.

- Two points can be fitted perfectly by a line.
- The residuals are zero.
- Nothing is left to estimate the noise.

Residuals Are the Gaps from the Fitted Curve

The residual for measurement i is

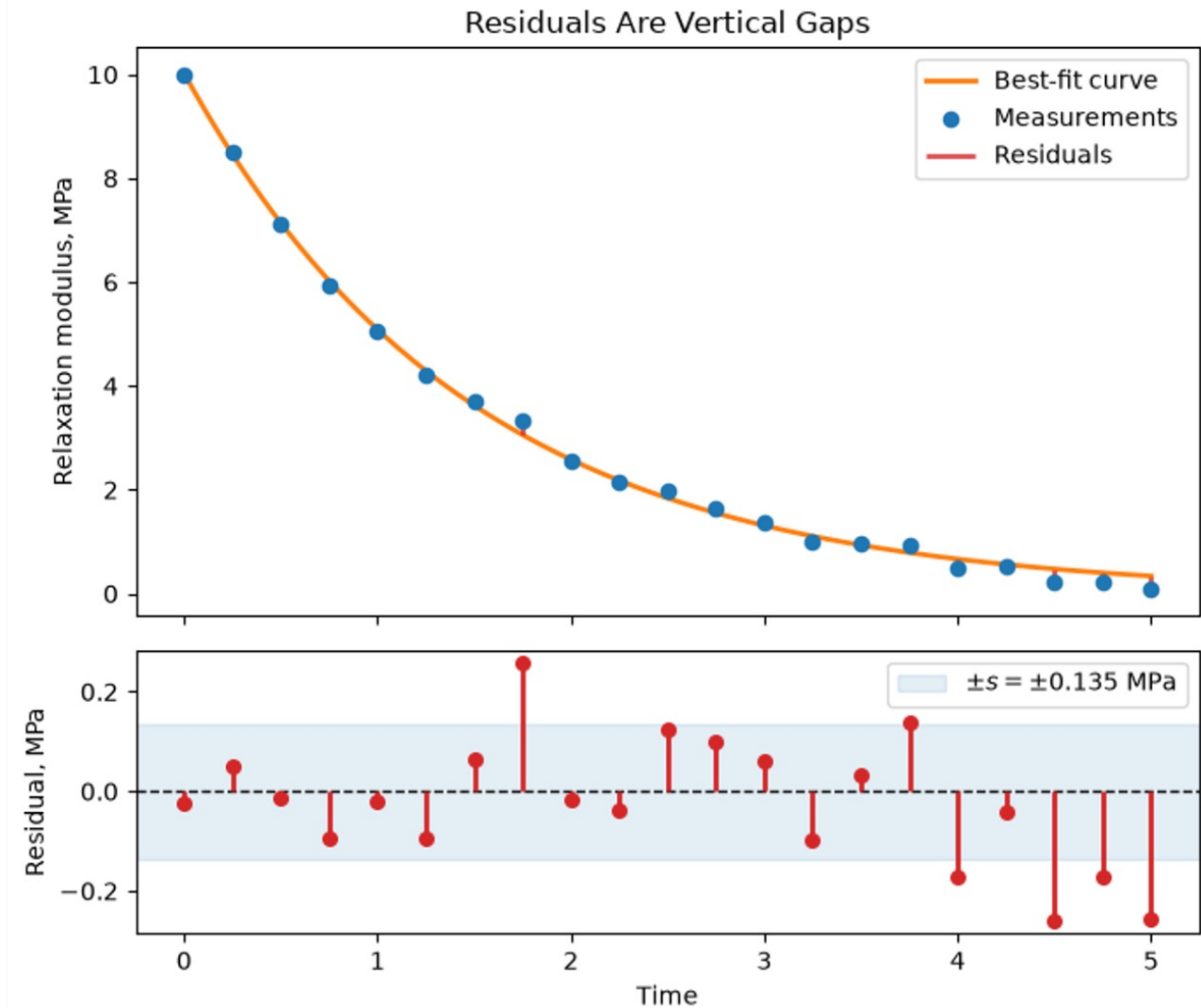
$$e_i = G_i - \hat{G}_0 \exp\left(-\frac{t_i}{\hat{\tau}}\right)$$

The residual sum of squares is

$$\text{SSE} = \sum_{i=1}^n e_i^2$$

Estimate the **noise variance** with

$$s^2 = \frac{\text{SSE}}{n - p}$$



Two experiments use the same model and measurement times. Experiment B has larger residuals. Which experiment should have greater parameter uncertainty?

- A. Experiment A
- B. Experiment B
- C. They should be the same
- D. It depends only on the optimizer

B. Larger residuals imply more estimated noise, so the fitted parameters can move more.

Covariance Measures Parameter Uncertainty

For a linear regression model,

$$\mathbf{y} = \mathbf{X}\boldsymbol{\theta} + \boldsymbol{\varepsilon}$$

the least-squares estimate is

$$\hat{\boldsymbol{\theta}} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

Its estimated covariance matrix is

$$\widehat{\text{Cov}}(\hat{\boldsymbol{\theta}}) = s^2 (\mathbf{X}^T \mathbf{X})^{-1}$$

For parameter j ,

$$\text{SE}(\hat{\theta}_j) = \sqrt{\left[\widehat{\text{Cov}}(\hat{\boldsymbol{\theta}})\right]_{jj}}$$

- s^2 : how noisy the measurements are
- $\mathbf{X}^T \mathbf{X}$: how informative the experiment is
- diagonal entries: uncertainty of individual parameters
- off-diagonal entries: how parameter estimates move together

$$\text{Cov} \left(\begin{bmatrix} \hat{\beta}_0 \\ \hat{\beta}_1 \end{bmatrix} \right) = \begin{bmatrix} \text{Var}(\hat{\beta}_0) & \text{Cov}(\hat{\beta}_0, \hat{\beta}_1) \\ \text{Cov}(\hat{\beta}_0, \hat{\beta}_1) & \text{Var}(\hat{\beta}_1) \end{bmatrix}$$

Measurement Range Controls Slope Precision

For the slope in simple linear regression,

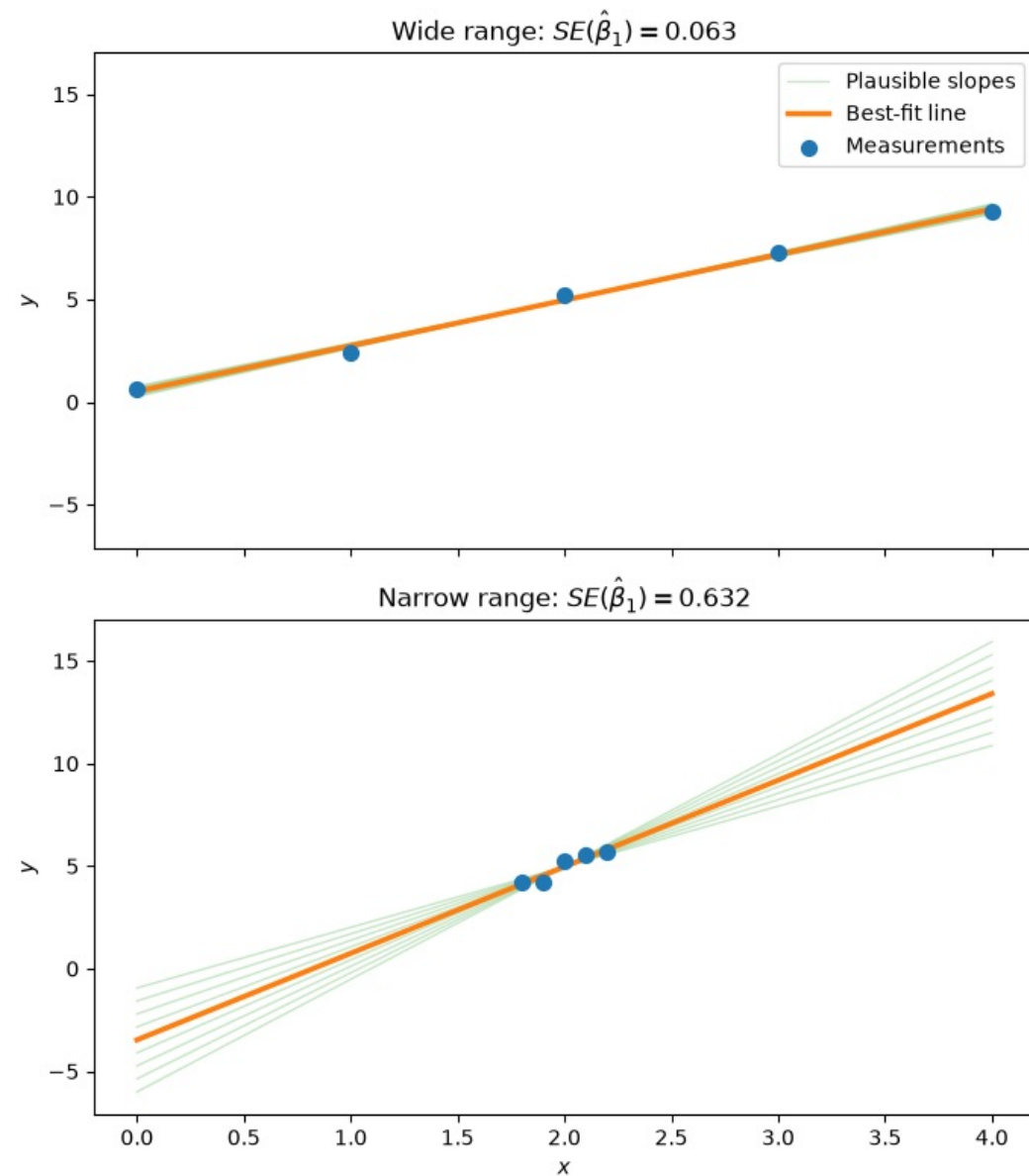
$$SE(\hat{\theta}_1) = \frac{s}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2}}$$

Assume the measurement noise is

$$s = 0.20$$

	Wide range	Narrow range
x values	0, 1, 2, 3, 4	1.8, 1.9, 2.0, 2.1, 2.2
$\sum (x_i - \bar{x})^2$	10.0	0.10
Slope SE	0.063	0.632

The same number of measurements give a ten-times more precise slope when x spans a useful range.



What a 95% Confidence Interval Means

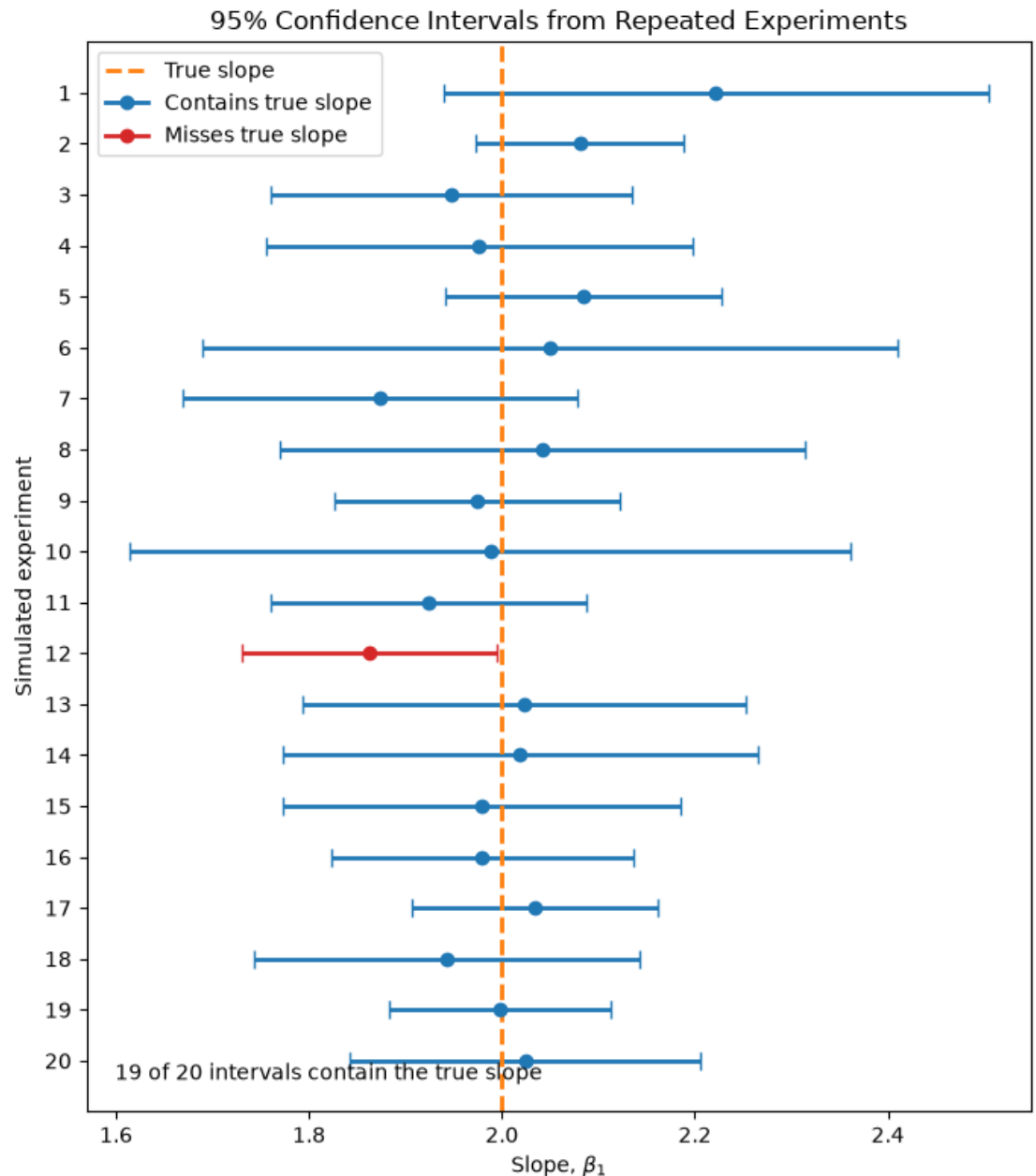
A 95% confidence interval for a linear-regression parameter is

$$\hat{\theta}_j \pm t_{0.975, n-p} \text{SE}(\hat{\theta}_j)$$

If we repeated the complete experiment many times, approximately 95% of the intervals constructed this way would contain the true parameter.

It does not mean

- 95% of the measurements lie inside the interval.
- The parameter changes from experiment to experiment.
- There is a 95% probability that this particular fixed parameter lies inside this particular interval.



Confidence and Prediction Intervals Answer Different Questions

Define

$$h_{\text{new}} = \mathbf{x}_{\text{new}}^T (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{x}_{\text{new}}$$

Uncertainty in the mean response:

$$\hat{y}_{\text{new}} \pm t_{0.975, n-p} s \sqrt{h_{\text{new}}}$$

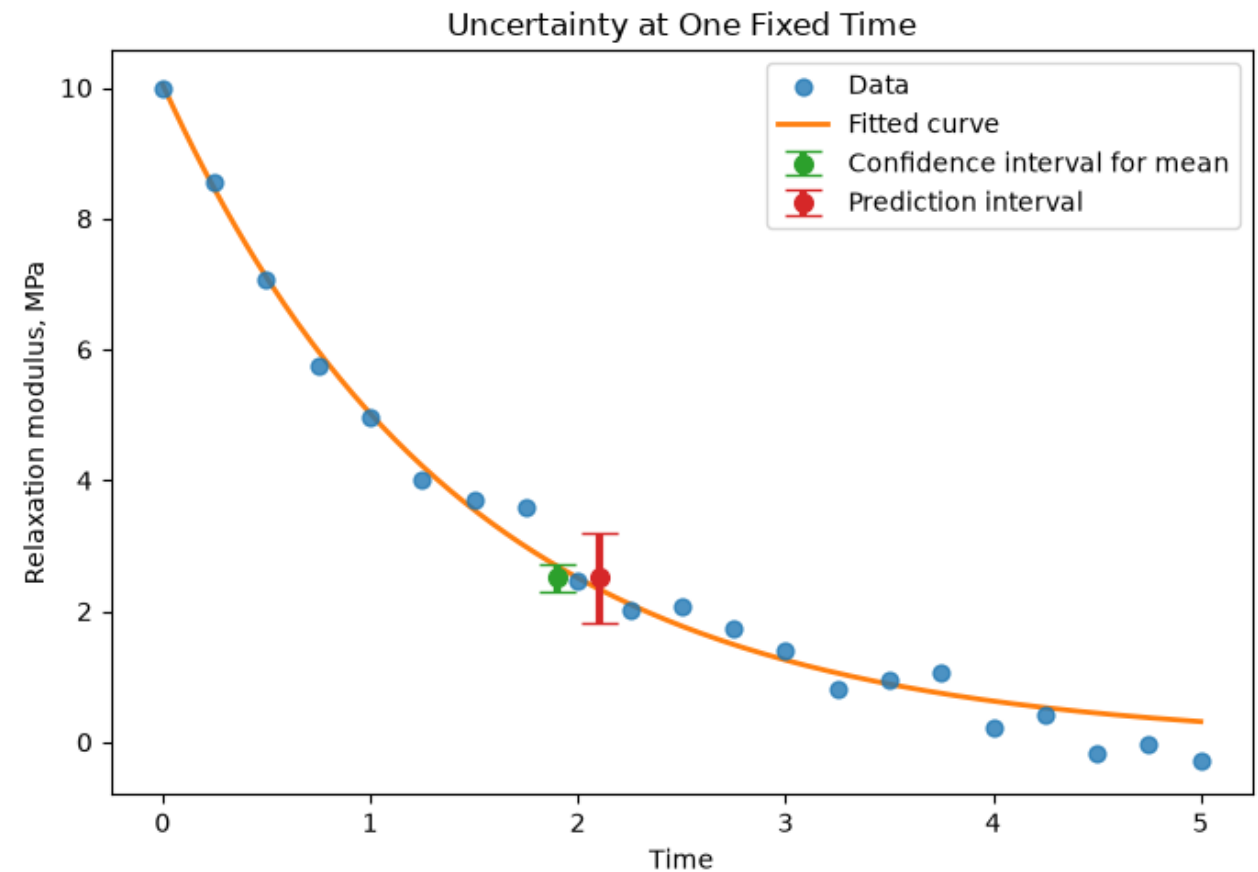
Uncertainty for a new measurement:

$$\hat{y}_{\text{new}} \pm t_{0.975, n-p} s \sqrt{1 + h_{\text{new}}}$$

- **confidence interval:** uncertainty in the road's centerline;
- **prediction interval:** region where the next car may be located.

Why is the prediction interval wider?

It is wider because it includes both uncertainty in the fitted curve and the noise of a new measurement.



A Nonlinear Model Is Locally Linear Near the Fit

Let

$$\boldsymbol{\theta} = \begin{bmatrix} G_0 \\ \tau \end{bmatrix}$$

A small parameter change gives

$$\mathbf{f}(\hat{\boldsymbol{\theta}} + \Delta\boldsymbol{\theta}) \approx \mathbf{f}(\hat{\boldsymbol{\theta}}) + \mathbf{J}\Delta\boldsymbol{\theta}$$

The Jacobian is

$$J_{ij} = \left. \frac{\partial f(t_i, \boldsymbol{\theta})}{\partial \theta_j} \right|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}}$$

$$\widehat{\text{Cov}}(\hat{\boldsymbol{\theta}}) \approx s^2 (\mathbf{J}^T \mathbf{J})^{-1}$$

The Jacobian measures how the predicted curve moves when each parameter moves.

Mathematically,

$$\text{pcov} \approx \widehat{\text{Cov}}(\hat{\boldsymbol{\theta}}) \quad \text{SE}(\hat{\theta}_j) = \sqrt{\text{pcov}_{jj}}$$

```
popt, pcov = curve_fit(
    maxwell_curve,
    t,
    G_simulation,
    p0=[8.0, 2.5],
    bounds=(0, np.inf),
)

parameter_se = np.sqrt(np.diag(pcov))

print(popt)
print(parameter_se)

[10.02239089  1.47395005]
[0.09549235  0.02189633]
```

From SciPy Covariance to Parameter Intervals

Let

$$\boldsymbol{\theta} = \begin{bmatrix} G_0 \\ \tau \end{bmatrix}$$

A small parameter change gives

$$\mathbf{f}(\hat{\boldsymbol{\theta}} + \Delta\boldsymbol{\theta}) \approx \mathbf{f}(\hat{\boldsymbol{\theta}}) + \mathbf{J}\Delta\boldsymbol{\theta}$$

The Jacobian is

$$J_{ij} = \left. \frac{\partial f(t_i, \boldsymbol{\theta})}{\partial \theta_j} \right|_{\boldsymbol{\theta}=\hat{\boldsymbol{\theta}}}$$

$$\widehat{\text{Cov}}(\hat{\boldsymbol{\theta}}) \approx s^2 (\mathbf{J}^\top \mathbf{J})^{-1}$$

$$t_{0.975,19} = 2.093 \quad \hat{\theta}_j \pm 2.093 \text{ SE}(\hat{\theta}_j)$$

```
popt, pcov = curve_fit(
    maxwell_curve,
    t,
    G_simulation,
    p0=[8.0, 2.5],
    bounds=(0, np.inf),
)

parameter_se = np.sqrt(np.diag(pcov))

print(popt)
print(parameter_se)
```

[10.02239089 1.47395005]
[0.09549235 0.02189633]

Parameter	Estimate	Standard Error	Approximate 95% CI
G_0	10.022 MPa	0.095 MPa	[9.823, 10.222] MPa
τ	1.474 s	0.0219 s	[1.428, 1.520] s

At one fixed time, why is the prediction interval wider than the confidence interval for the mean?

- A. It uses more fitted parameters
- B. It includes both curve uncertainty and noise in a new measurement
- C. The true parameter changes
- D. The optimizer is nonlinear

B.

Interactive Python Time

07_regression_uncertainty.ipynb